

Temeraire: Redis-Reproduktion mit öffentlichem Code

Von der hugepage-aware Idee bis zum funktionierenden Experiment

Aldo Costa

Heinrich-Heine-Universität Düsseldorf

Juli 2026

Was wird hier reproduziert?

Temeraire

Hugepage-bewusster Backend-Allocator für
TCMalloc

Hunter et al., OSDI 2021

- Ziel im Paper: weniger TLB-Druck durch bessere Platzierung
- Ziel hier: der öffentliche Redis 6.0.9-Case-Study-Teil
- Kein Google-Fleet-Experiment, keine Produktions-Telemetrie

Arbeitsfrage

Kann die Redis-Methode mit öffentlichem Code und lokaler Hardware so nachgebaut werden, dass die kleine Effektgröße des Papers sichtbar wird?

Hugepages als Ausgangspunkt

- Normale Seiten: 4 KiB
- Hugepages: 2 MiB, also 512 normale Seiten
- Ein Hugepage-TLB-Eintrag deckt viel mehr Speicher ab
- Der Gewinn ist weniger Adressübersetzung, nicht ein größerer Cache

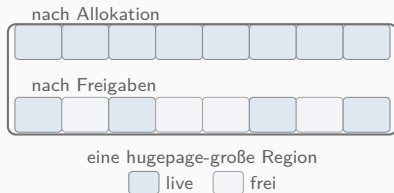
Problem

Hugepages helfen nur, wenn der Speicher passend ausgerichtet und zusammenhängend bleibt.

Allocator-Rolle

`malloc` entscheidet indirekt, ob live Objekte eine Hugepage erhalten, fragmentieren oder für die Freigabe blockieren.

Fragmentierung innerhalb einer Hugepage

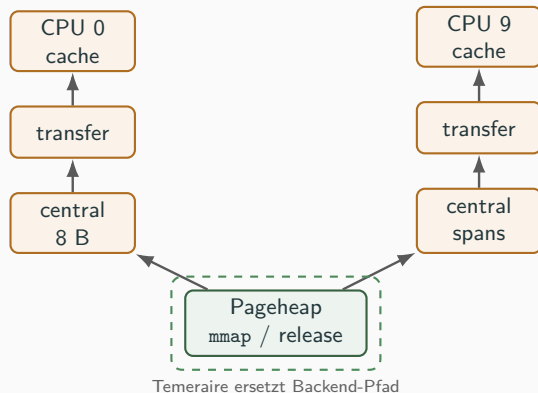


- Freier Speicher reicht nicht aus, wenn er zwischen live Objekten liegt
- Teilfreigabe kann RAM sparen, aber Hugepage-Abdeckung zerstören
- Nichts freigeben kann TLB-Abdeckung halten, kostet aber Speicher

Temeraire macht diesen Trade-off explizit.

Es packt kleine Objekte bewusst auf schon teilgefüllte Hugepages.

Wo Temeraire in TCMalloc eingreift



- Obere Caches bleiben TCMalloc
- Temeraire ersetzt den Backend-Pageheap
- Reproduktion vergleicht deshalb Legacy Pageheap gegen den hugepage-aware Pfad

Vier Komponenten teilen die Anfragen nach Größe auf

HugeAllocator



holt ausgerichtete Hugepage-Bereiche vom System

HugeCache



hält leere, bereits backed Hugepages zur Wiederverwendung

HugeFiller



packt kleine Allokationen auf teilbelegte Hugepages

HugeRegion



behandelt mittlere Allokationen mit größerem Rest

Routing: $< 1 \text{ MiB} \rightarrow \text{HugeFiller}$; $1 \text{ MiB} - 1 \text{ GiB} \rightarrow \text{HugeCache} / \text{HugeRegion}$; $\geq 1 \text{ GiB}$

$\rightarrow \text{HugeCache}$. Redis trifft vor allem den kleinen Pfad.

Workload aus dem Paper

- Redis 6.0.9
- LPUSH von fünf Werten
- LRANGE 0 4
- 2000 Trials, je 1 Mio. Requests

Erwarteter Effekt

Redis erzeugt viele kleine Listen- und String-Allokationen. Wenn `HugeFiller` sie besser platziert, sinkt der TLB-Druck.

$$\text{kombiniert} = \frac{2}{1/r_{\text{LPUSH}} + 1/r_{\text{LRANGE}}}$$

Konsequenz für die Messung

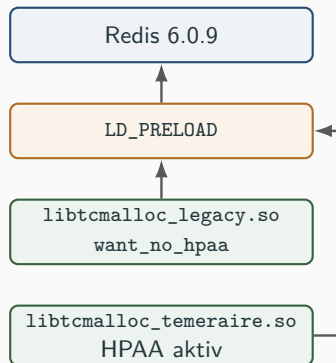
Der erwartete Unterschied liegt unter einem Prozent. Reihenfolge, THP-Zustand und Laufzeit-Validierung dürfen nicht implizit bleiben.

Zwei Libraries machen den Vergleich möglich

Rolle der Wrapper

Redis wird nicht zweimal umgebaut. Stattdessen werden zwei kleine TCMalloc-Shared-Libraries gebaut und beim Start in den Prozess geladen.

- gleicher Redis-Build
- gleiche Wrapper-Quelle
- unterschiedliche TCMalloc-Backend-Konfiguration



Abgrenzung

Der Wrapper ist kein neuer Allocator. Er macht den historischen TCMalloc-Code preloadbar, damit der Legacy-vs.-Temeraire-Pfad messbar wird.

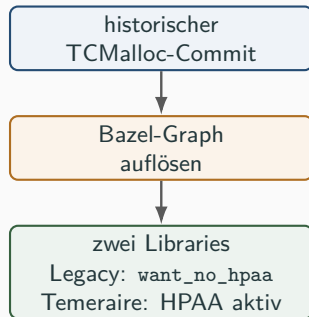
Der Wrapper musste in den historischen Build-Graph

Problem

Der Vergleich sollte nicht an selbst nachgebauten Link-Flags oder unvollständigen TCMalloc-Abhängigkeiten hängen.

- TCMalloc ist selbst ein Bazel-Projekt
- Abseil und interne Targets kommen aus dem nativen Build-Graph
- `want_no_hpaa` ist ein TCMalloc-Target, kein Runtime-Flag

Bazel brachte TCMallocs Build-Graphen mit: Abseil, interne Targets und Link-Semantik mussten nicht in CMake nachgebaut werden.



Startpunkt: falsche Nähe zum Paper

Vorher

- Redis 7.2.5 statt Redis 6.0.9
- glibc gegen gperftools
- kein Legacy-Pageheap gegen Temeraire
- Scope war anfangs zu breit formuliert

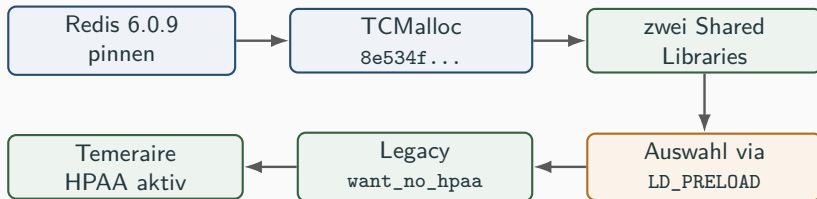
Entscheidung

Das Artefakt wird auf den Redis-Case-Study-Teil eingegrenzt. Die Fleet- und Rollout-Ergebnisse bleiben außerhalb der Reproduktion.

Erster Fix

Erst die Behauptung begrenzen, dann die Technik bauen.

Umbau 1: Den richtigen Vergleich bauen



- Der historische Commit enthält noch den Schalter für den Legacy-Pfad
- Beide Varianten kommen aus derselben Wrapper-Quelle
- Die intendierte Differenz ist der TCMalloc-Backend-Pfad

Was nicht sauber funktionierte

- Moderner TCMalloc-Checkout passte nicht zur Wrapper-Strategie
- Externer Bazel-Wrapper erbte nicht alle Dependencies
- `want_no_hpaa` war nicht global sichtbar

Was am Ende funktionierte

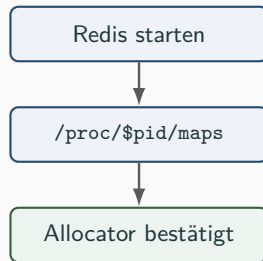
- Wrapper in `tcmalloc/testing`
- Bazelisk und Bazel-Version pinnen
- LLVM-Pfad durch Make, Autotools und Bazel reichen
- fehlende Release-Symbole dynamisch auflösen

Ein erfolgreicher Start beweist noch nicht den Allocator

- Redis kann weiter `mem_allocator: libc` melden
- Diese Meldung kommt aus der Redis-Build-Konfiguration
- Für `LD_PRELOAD` zählt, was der Prozess wirklich mappt

Prüfung

`/proc/$pid/maps` muss die passende Library zeigen:
`libtcmalloc_legacy.so` oder
`libtcmalloc_temeraire.so`.



Ohne aktive Hugepages misst der Lauf nicht Temeraire

Beobachtung in frühen Runs

```
AnonHugePages: 0kB
```

Redis lief, die Allocator-Varianten liefen, aber der Hugepage-Mechanismus war nicht belegt.

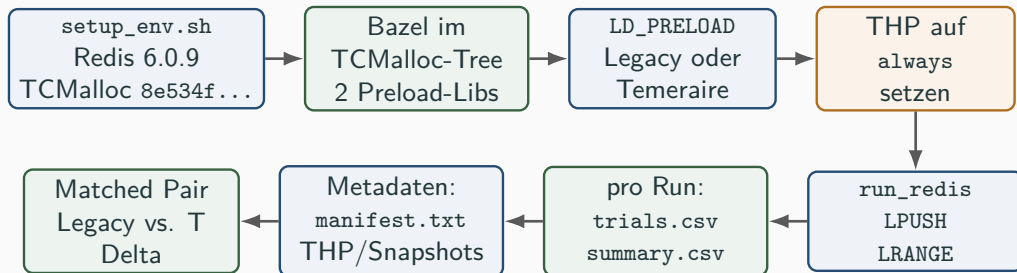
Fix im Harness

- `thp-before.txt` und `thp-after.txt`
- `memory-sample-0250..2000.txt`
- `smaps_rollup` mit `AnonHugePages`
- nur THP-aktive Runs für das Paper-Signal verwenden

Kriterium für gültige Runs

Spätere Runs liefen mit `[always] madvise never`; Redis-Snapshots zeigten 14–18 MiB `AnonHugePages` im Prozess. Erst dann war der Hugepage-Pfad wirklich Teil der Messung.

Der gültige Lauf prüft Aufbau, Mechanismus und Ergebnis



Konfiguration

- 2000 Trials; je 1 Mio. Requests
- 50 Clients, Pipeline 16
- ein Redis-Binary, zwei TCMalloc-.sos



Auswertung

- LPUSH/LRANGE als harmonisches Mittel
- THP und AnonHugePages protokolliert
- Delta nur im passenden Legacy/Tembraire-Paar

Release-on war faktisch nicht eingeschaltet

▶ Was lief

- Background-Thread gestartet
- Per-CPU-Caches wurden gepflegt

⚠ Was fehlte

Release-Rate nicht gesetzt: TCMalloc blieb bei 0 B/s. Damit gab es keine periodische Pageheap-Freigabe.

Konsequenz

Die alten Werte waren kein Release-on-Ergebnis. Der korrigierte Lauf setzt eine positive Rate explizit; das Paper nennt seine Rate nicht.

Release-off liegt nahe am kleinen Paper-Effekt

Run	Reihenfolge	Delta
THP fixed-order	L first	+1.88%
balanced 1	L first	+0.43%
balanced 2	T first	+0.48%
balanced 3	L first	-0.76%
balanced 4	T first	+3.46%

Befund

Vier von fünf THP-aktiven vollständigen Runs favorisieren Temeraire. Der Median liegt bei +0.48%.

Paper-Vergleich

Redis[†] im Paper: +0.75%
Periodic release disabled.

Die alten Release-on-Runs bleiben nur als Fehlerprotokoll



release-on
effektiv 0 B/s

Kein Paper-Vergleich

Der Manifestname versprach eine Bedingung, die der Prozess nicht erfüllte.

Warum die Logs bleiben

Sie dokumentieren den Konfigurationsfehler und die lokale Streuung. Die Deltas werden nicht als Ergebnis verwendet.

Drei Release-Raten ergeben keinen stabilen Trend

Rate	Pair 1	Pair 2	Pair 3	Pair 4	Median
16 MiB/s	+1.91%	-0.16%	-0.27%	-1.37%	-0.21%
64 MiB/s	-6.88%	+2.10%	+11.49%	+12.21%	+6.80%
256 MiB/s	+15.73%	-0.09%	+0.50%	-0.77%	+0.20%

Was jetzt stimmt

24 Allocator-Runs, je 2000 Trials. Jeder Redis-Log bestätigt die positive Release-Rate.

Was das Ergebnis begrenzt

Kein monotoner Rate-Effekt. Absolute Leistung fiel zeitweise um mehr als 20% und erholte sich später wieder.

Host-Drift ist größer als der erwartete Effekt

Auswertbarer Befund

Release-off liegt mit +0.48% Median im kleinen Effektbereich des Paper-Werts von +0.75%. Die fünf Run-Paare streuen jedoch deutlich.

Korrigierter Befund

Release-on ist technisch korrekt gemessen, zeigt aber keinen stabilen Rate-Trend. Host-Drift ist größer als der Paper-Effekt.

Vertretbare Aussage

Das Artefakt rekonstruiert den Redis-Vergleich mit öffentlichem Code. Der korrigierte Lauf zeigt vor allem die Grenze konsekutiver Ein-Stunden-Runs.

Was diese Reproduktion nicht zeigen kann

Nicht reproduziert

- Google-Fleet-Experiment
- Produktions-Rollout
- interne Allocator-Binaries
- exakte Paper-Hardware

Lokale Abweichungen

- Docker/WSL2 und geteilter Kernel
- Consumer i7 statt Skylake Xeon Server
- THP-Policy aus der Host/VM-Umgebung
- Public TCMalloc als historische Annäherung

Nutzen der Grenzen

Sie trennen den methodisch reproduzierten Redis-Pfad von den Teilen des Papers, die ohne Google-Infrastruktur nicht nachstellbar sind.

Technische Lektion

Der schwere Teil war nicht `redis-benchmark`, sondern der Weg zu einem echten Legacy-vs.-Temeraire-Vergleich mit aktiven Hugepages.

- Vergleich aus public TCMalloc rekonstruiert
- Preload zur Laufzeit validiert
- THP-Fehlerbild gefunden und instrumentiert
- Release-on korrigiert und mit drei Raten neu gemessen

Endstand

Release-off zeigt ein kleines positives Mediansignal bei hoher Streuung. Release-on ist aktiv, aber Host-Drift verhindert ein stabiles Ergebnis.